10-10-2026

CO1 -ASSSESMENT TOOL 1

APPLICATION BASED QUESTIONS

COURSE NAME: NATURAL LANGUAGE PROCESSING

COURSECODE: DSA0308

NAME: R SAI HASNIKA **REG NO: 192424178**

Section 1: Regular Expressions – Resume Information Extraction

Answer:

1. Importing the Regular Expression module: The statement import re is written at the beginning of the program because the re module provides functions such as re.search() and re.findall() that are required to extract information from resume text.

2. Storing the resume details: A multiline string is created using resume = """..."""" to store the candidate's name, email address, mobile number, technical skills, and experience. This provides sample resume data for the program to process.

3. Extracting the candidate's name: The function re.search(r"Name:\s*(.+)", resume) searches for the name after the label Name: The pattern \s* allows spaces, while (.+) captures the name. The method .group() retrieves the captured name.

4. Extracting the email address: The function re.findall(r'[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}', resume) identifies email addresses in the resume. This pattern checks for the username, @

symbol, domain, and extension. findall() is used because a resume may contain more than one email address.

5. Extracting the mobile number: The statement re.findall(r'\b\d{10}\b', resume) searches for mobile numbers containing exactly ten digits. Here, \d represents a digit, {10} specifies ten digits, and \b helps identify the boundaries of the number.

6. Defining the technical skills: A list containing Python, Java, SQL, Machine Learning, and NLP is created. This list is required because the program must check the resume for each of the technical skills mentioned in the question.

7. Detecting the skills in the resume: A for loop checks each skill using re.search(). The re.IGNORECASE option allows the program to recognize skills regardless of uppercase or lowercase letters. Whenever a skill is found, skills.append(skill) adds it to the result list.

8. Extracting years of experience: The statement re.search(r'(\d+)\s+years?\s+of\s+experience', resume, re.IGNORECASE) searches for the number of years mentioned in the resume. The pattern (\d+) captures the numerical value, while years? allows both singular and plural forms.

9. Converting experience into a number: The captured experience is converted into an integer using int(experience_match.group(1)). This conversion is necessary because the program must compare the experience with the minimum requirement of two years. If no matching experience is found, the program can assign zero.

10. Displaying the candidate profile: The print() statements display the candidate's name, email, mobile number, technical skills, and experience. This step is necessary to present the extracted information in an organized and readable format.

11. Checking the eligibility criteria: The condition if experience >= 2 and "Python" in skills: checks whether the candidate has at least two years of experience and possesses Python skills. The and operator ensures that both conditions must be satisfied for the candidate to qualify.

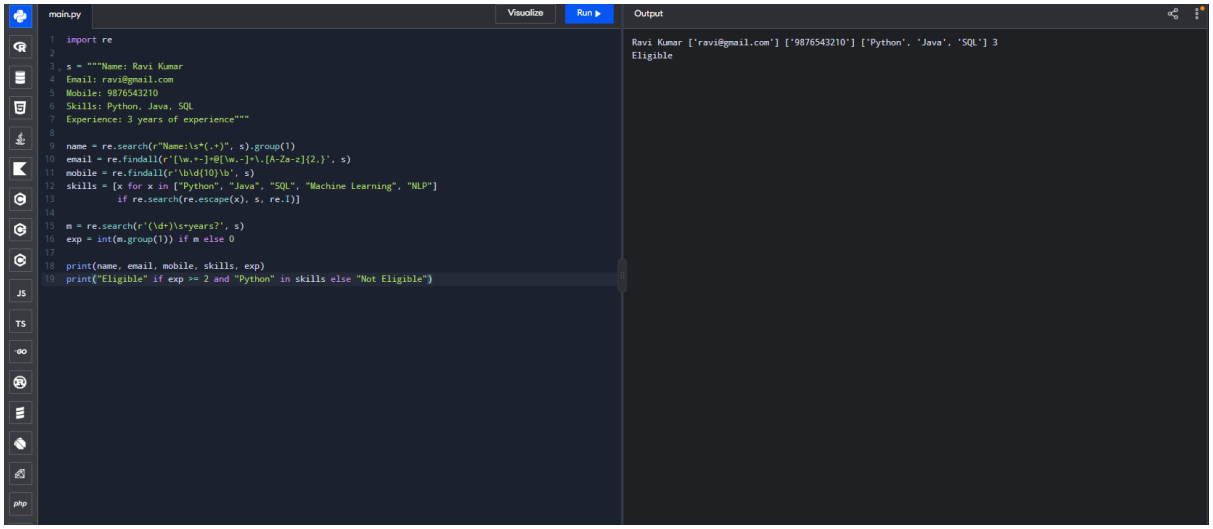
12. Displaying the final result: The if-else statement displays whether the candidate is eligible for shortlisting. If both eligibility conditions are satisfied, the candidate is marked as eligible; otherwise, the candidate is marked as ineligible. This helps the HR department identify suitable candidates quickly.

Code :

```
import re
s = """Name: Ravi Kumar
Email: ravi@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL
Experience: 3 years of experience"""
name = re.search(r"Name:\s*(.+)", s).group(1)
email = re.findall(r'[\w.+-]+@[\w.-]+\.[A-Za-z]{2,}', s)
mobile = re.findall(r'\b\d{10}\b', s)
skills = [x for x in ["Python", "Java", "SQL", "Machine Learning", "NLP"]
        if re.search(re.escape(x), s, re.I)]
m = re.search(r'(\d+)\s+years?', s)
exp = int(m.group(1)) if m else 0
print(name, email, mobile, skills, exp)
print("Eligible" if exp >= 2 and "Python" in skills else "Not Eligible")
```

Expected Output:

```
CANDIDATE PROFILE
Name: Ravi Kumar
Email: ['ravi@gmail.com']
Mobile: ['9876543210']
Skills: ['Python', 'Java', 'SQL', 'Machine Learning']
Experience: 3 years
Status: Eligible for shortlisting
```



Section 2: Product Search System

Answer :

1. Importing the Regular Expression module: The statement import re is used to access Python's Regular Expression functions. These functions are necessary to search product names using exact, prefix, suffix, and partial keyword matching.

2. Creating the product list: A list named products is created to store the names of products available in the e-commerce store. This list acts as the collection through which the search operations are performed.

3. Defining the search keyword: The variable keyword = "apple" stores the word that the user wants to search for. The statement pattern = re.escape(keyword) prepares the keyword for regex matching by treating special regex characters as literal characters.

4. Performing exact keyword searching: The function re.fullmatch(pattern, p, re.IGNORECASE) checks whether the entire product name matches the keyword. It is used for exact searching because a product such as Apple iPhone should not match the exact keyword apple.

5. Performing prefix-based searching: The pattern r"^" + pattern is used with re.search() to find products whose names begin with the keyword. The ^ symbol represents the beginning of the string, so products such as Apple Watch are included.

6. Performing suffix-based searching: The pattern pattern + r"$" checks whether the product name ends with the keyword. The $ symbol represents the end of the string, so a product such as Green apple would match the suffix search for apple.

7. Performing partial keyword searching: The function re.search(pattern, p, re.IGNORECASE) checks whether the keyword appears anywhere in a product name. This is useful when customers remember only part of a product name, such as finding Pineapple Juice by searching for apple.

8. Applying case-insensitive matching: The flag re.IGNORECASE is included in the search functions so that Apple, APPLE, and apple are treated as equivalent. This makes the search system more flexible and convenient for customers.

9. Storing matching products: Each search operation uses a list comprehension to examine all the products and store only those that satisfy the required pattern. Separate lists named exact, prefix, suffix, and partial keep the results organized.

10. Creating the search report: A dictionary named reports stores each search type along with its corresponding matching product list. This structure makes it easier to display the results of all search methods in one place.

11. Displaying matching products: A for loop using reports.items() accesses each search type and its results. The print() statements display the category and matching product names, allowing the user to compare the results of different searches.

12. Counting the matching products: The function len(matches) calculates the number of products found for each search. The program displays this count along with the matching names, completing the required product search report.

Code :

```
import re
import re


products = ["Apple", "Apple iPhone", "Apple Watch",
```

```
        "Samsung Phone", "Pineapple Juice"]
key = "apple"

patterns = {
    "Exact": lambda p: re.fullmatch(re.escape(key), p, re.I),
    "Prefix": lambda p: re.search("^" + re.escape(key), p, re.I),
    "Suffix": lambda p: re.search(re.escape(key) + "$", p, re.I),
    "Partial": lambda p: re.search(re.escape(key), p, re.I)
}

for name, check in patterns.items():
    result = [p for p in products if check(p)]
    print(name, result, "Count:", len(result))
```

Expected Output :

Exact Search

Matching products: ['Apple']

Total matches: 1

Prefix Search

Matching products: ['Apple', 'Apple iPhone', 'Apple Watch', 'apple laptop']

Total matches: 4

Suffix Search

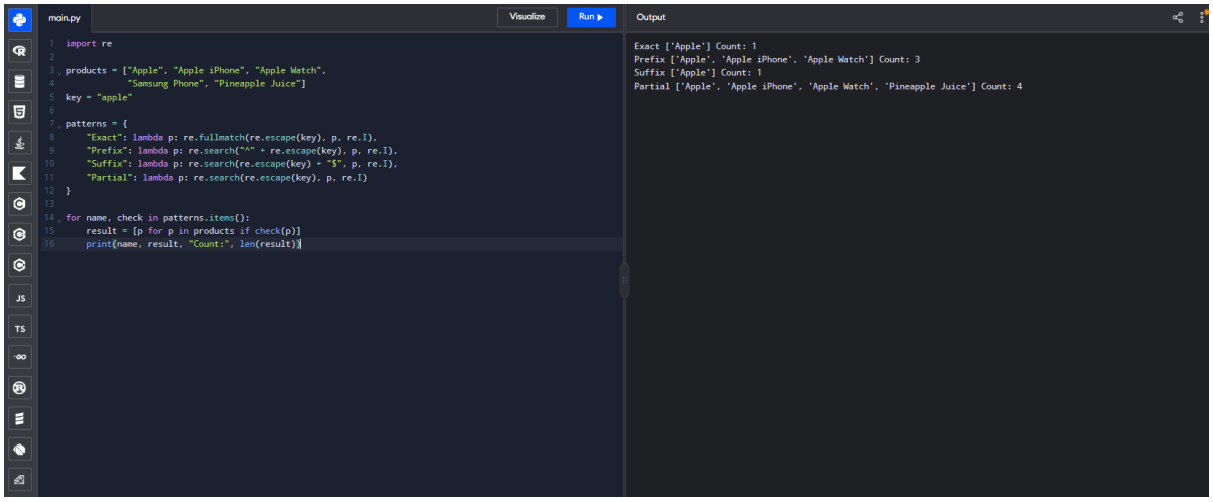
Matching products: ['Apple', 'Pineapple Juice']

Total matches: 2

Partial Search

Matching products: ['Apple', 'Apple iPhone', 'Apple Watch', 'Pineapple Juice', 'apple laptop']

Total matches: 5



Section 3: University Registration System

Answer :

1. Importing the Regular Expression module: The statement import re is written at the beginning because the registration system needs to check whether student information follows specific formats using Regular Expressions.

2. Storing student information: Variables such as reg_no, email, course, semester, and mobile are created to store the student's details. These values are used as sample inputs for demonstrating the validation process.

3. Defining the register number pattern: The pattern r"\d{9}" is created to check whether the student register number contains exactly nine digits. This pattern is used because the example university requires a nine-digit register number.

4. Defining the institutional email pattern: The pattern r"[\w.+-]+@university\.edu" checks whether the email has a username followed by the specified institutional domain. The escaped dot \. is used because a dot has a special meaning in Regular Expressions and must be matched literally.

5. Defining the course code pattern: The pattern r"[A-Z]{3}\d{3}" checks whether the course code contains three uppercase letters followed by three digits. For example, CSE101 matches the pattern, while a code with a different structure does not.

6. Defining the mobile number pattern: The pattern r"\d{10}" checks whether the mobile number contains exactly ten digits. It is used to reject numbers that contain fewer or more digits than the required format.

7. Initializing the validation status: The statement valid = True assumes that the registration is valid at the beginning. If any field fails its validation check, the variable is changed to False, allowing the program to track the overall result.

8. Validating the register number: The condition re.fullmatch(reg_pattern, reg_no) checks whether the complete register number matches the required pattern. The program displays a valid or invalid message and sets valid = False if the input does not match.

9. Validating the email and course code: The program uses re.fullmatch() with email_pattern and course_pattern to verify the complete email address and course code. Separate if-else statements display appropriate messages and update the overall validation status when necessary.

10. Validating the semester: The condition re.fullmatch(r"[1-8]", semester) checks whether the semester is a single digit between 1 and 8. This pattern is used because the example university permits only semesters within that range.

11. Displaying field validation messages: For each field, the program prints a message indicating whether the entered information is valid or invalid. Separate checks are necessary so that students can identify exactly which details need correction instead of receiving only a general error message.

12. Generating the final registration report: The final if-else statement checks the value of valid. If all five fields pass validation, the program displays Registration Successful; if at least one field fails, it displays Registration Unsuccessful. This provides a clear summary of the student's registration status.

Note: The register number, email domain, course code, and semester patterns are examples. In an actual university system, they should be adjusted to match the institution's official rules.

Code :

```
import re

reg = "192424178"
email = "student@university.edu"
course = "CSE101"
sem = "5"
mobile = "9876543210"

checks = {
    "Register Number": (r"\d{9}", reg),
    "Email": (r"[\w.+-]+@university\.edu", email),
    "Course Code": (r"[A-Z]{3}\d{3}", course),
    "Semester": (r"[1-8]", sem),
    "Mobile": (r"\d{10}", mobile)
}

valid = True

for name, (pattern, value) in checks.items():
    ok = bool(re.fullmatch(pattern, value))
    print(name, "Valid" if ok else "Invalid")
    if not ok:
        valid = False

print("Registration Successful" if valid
      else "Registration Unsuccessful")
```

Expected Output:

Register Number: Valid

Email: Valid

Course Code: Valid

Semester: Valid

Mobile Number: Valid

FINAL REGISTRATION REPORT

Registration Successful

